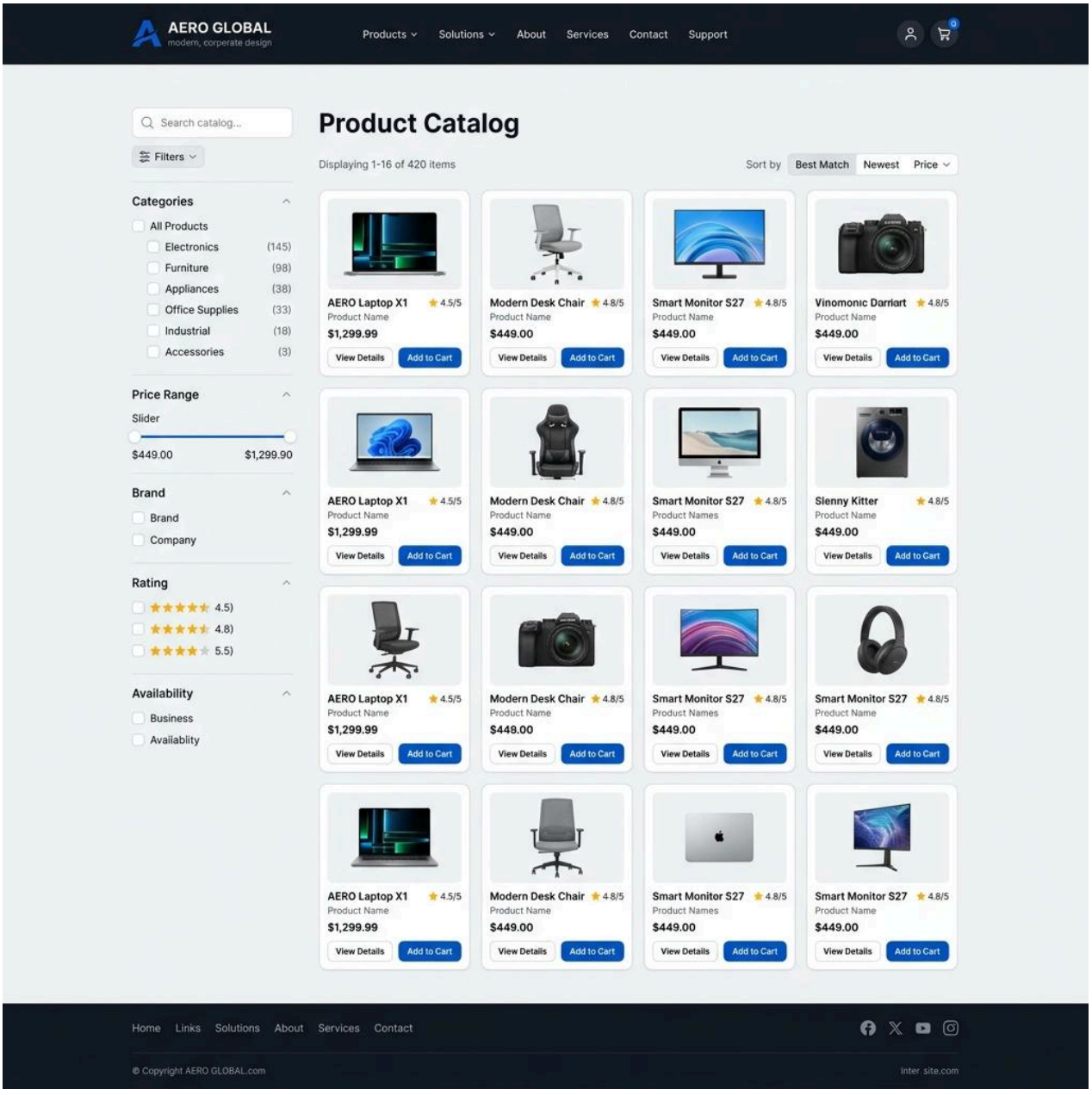
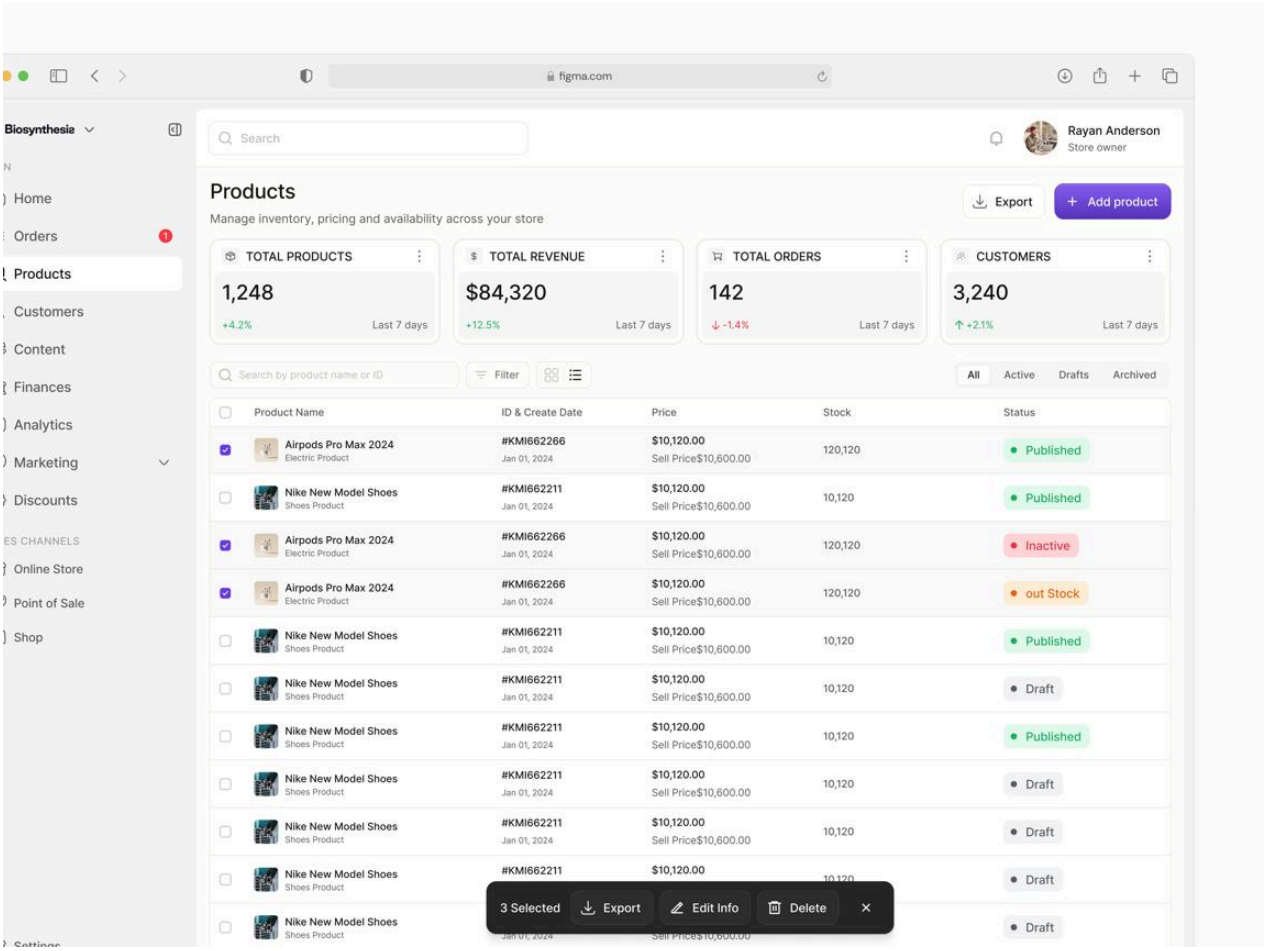
ფინალური პროექტი C# / .NET პროგრამირების კურსისთვის

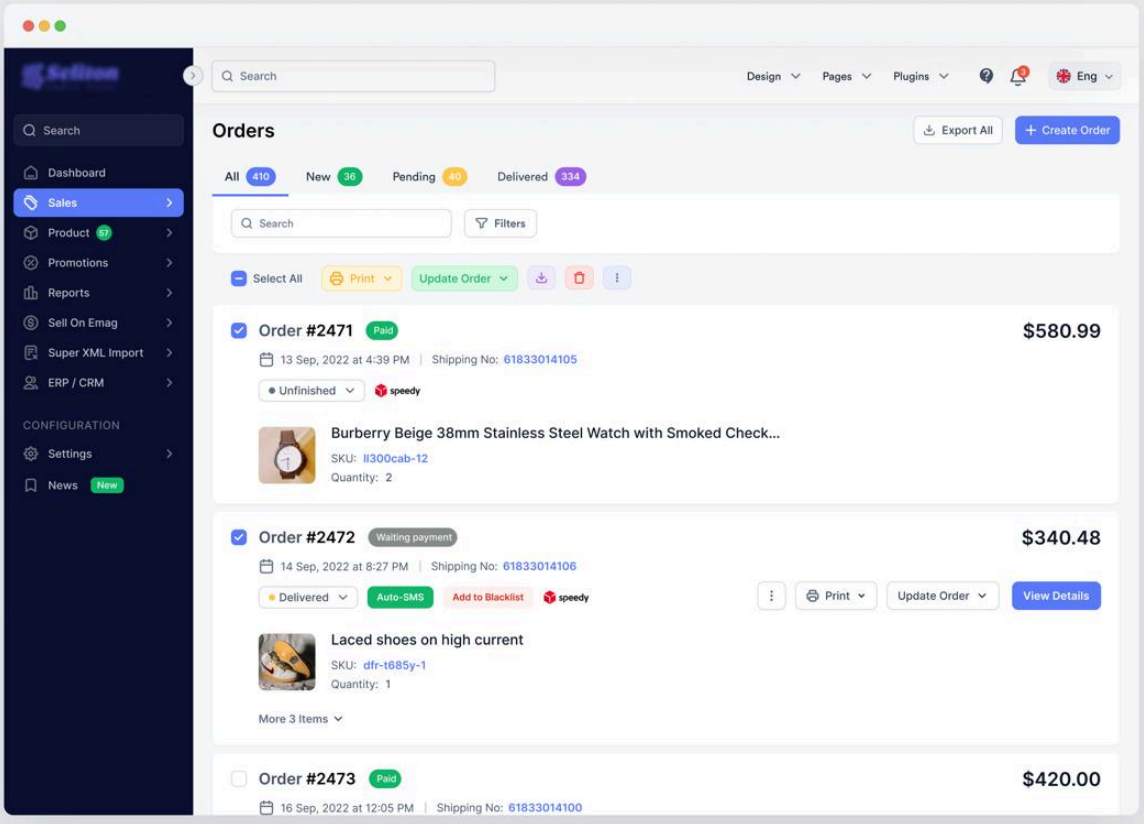
პროექტი აერთიანებს სილაბუსის ორივე ნაწილში შესწავლილ თემებს: C#-ის საფუძვლებს, OOP-ს, SOLID პრინციპებს, მონაცემთა ბაზებთან მუშაობას, Entity Framework-ს, ASP.NET Core Web API-ს, HTTP მოთხოვნებს, Dependency Injection-სა და ასინქრონულ პროგრამირებას.

პროექტი განკუთვნილია სტუდენტებისთვის, რომლებსაც კურსის დასრულების შემდეგ უნდა შეეძლოთ არა მხოლოდ ცალკეული დავალებების შესრულება, არამედ სრულფასოვანი, მონაცემთა ბაზასთან დაკავშირებული Web API აპლიკაციის შექმნა.

პროექტის თემა — ონლაინ მაღაზიის მართვის სისტემა




figma.com
Biosynthesia
Search
Rayan Anderson
Store owner
Home
Orders
Products
Customers
Content
Finances
Analytics
Marketing
Discounts
ES CHANNELS
Online Store
Point of Sale
Shop
Products
Manage inventory, pricing and availability across your store
Export
+ Add product
TOTAL PRODUCTS
1,248
+4.2%
Last 7 days
TOTAL REVENUE
$84,320
+12.5%
Last 7 days
TOTAL ORDERS
142
-1.4%
Last 7 days
CUSTOMERS
3,240
+2.1%
Last 7 days
Search by product name or ID
Filter
All
Active
Drafts
Archived
Product Name
ID & Create Date
Price
Stock
Status
Airpods Pro Max 2024
Electric Product
#KMI662266
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
120,120
Published
Nike New Model Shoes
Shoes Product
#KMI662211
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
10,120
Published
Airpods Pro Max 2024
Electric Product
#KMI662266
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
120,120
Inactive
Airpods Pro Max 2024
Electric Product
#KMI662266
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
120,120
out Stock
Nike New Model Shoes
Shoes Product
#KMI662211
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
10,120
Published
Nike New Model Shoes
Shoes Product
#KMI662211
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
10,120
Draft
Nike New Model Shoes
Shoes Product
#KMI662211
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
10,120
Published
Nike New Model Shoes
Shoes Product
#KMI662211
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
10,120
Draft
Nike New Model Shoes
Shoes Product
#KMI662211
Jan 01, 2024
$10,120.00
Sell Price$10,600.00
10,120
Draft
Nike New Model Shoes
Shoes Product
#KMI662211
Jan 01, 2024
$10,120.00
10,120
Draft
Nike New Model Shoes
Shoes Product
Draft
3 Selected
Export
Edit Info
Delete


Search
Design
Pages
Plugins
Eng
Search
Dashboard
Sales
Product
Promotions
Reports
Sell On Emag
Super XML Import
ERP / CRM
CONFIGURATION
Settings
News
New
Orders
Export All
+ Create Order
All 410
New 36
Pending 40
Delivered 334
Search
Filters
Select All
Print
Update Order
Order #2471
Paid
$580.99
13 Sep, 2022 at 4:39 PM | Shipping No: 61833014105
Unfinished
speedy
Burberry Beige 38mm Stainless Steel Watch with Smoked Check...
SKU: II300cab-12
Quantity: 2
Order #2472
Waiting payment
$340.48
14 Sep, 2022 at 8:27 PM | Shipping No: 61833014106
Delivered
Auto-SMS
Add to Blacklist
speedy
Print
Update Order
View Details
Laced shoes on high current
SKU: dfr-t685y-1
Quantity: 1
More 3 Items
Order #2473
Paid
$420.00
16 Sep, 2022 at 12:05 PM | Shipping No: 61833014100

პროექტის მიზანია ონლაინ მაღაზიის backend სისტემის შექმნა, რომელიც უზრუნველყოფს პროდუქტების, კატეგორიების, მომხმარებლებისა და შეკვეთების მართვას.

სტუდენტმა უნდა შექმნას ASP.NET Core Web API, რომელიც მონაცემებს შეინახავს MS SQL Server-ის ბაზაში და საშუალებას მისცემს კლიენტ აპლიკაციას HTTP მოთხოვნების საშუალებით მიიღოს, დაამატოს, შეცვალოს და წაშალოს ინფორმაცია.

პროექტის ძირითადი ფუნქციონალი

- პროდუქტების დამატება, ნახვა, განახლება და წაშლა.
- კატეგორიების შექმნა და პროდუქტებთან დაკავშირება.
- მომხმარებლების რეგისტრაციის მსგავსი მონაცემების შენახვა.
- შეკვეთების შექმნა და შეკვეთაში პროდუქტების დამატება.
- შეკვეთის ჯამური ღირებულების გამოთვლა.
- მონაცემების ვალიდაცია და შეცდომების მართვა.
- მონაცემთა ბაზასთან მუშაობა Entity Framework Core-ით.
- ასინქრონული ოპერაციები `async` / `await` -ის გამოყენებით.
- კოდის ორგანიზება OOP-ისა და SOLID-ის პრინციპებით.
-

1. პროექტის მოთხოვნები

სტუდენტმა პროექტი უნდა შეასრულოს ეტაპობრივად. თითოეული ეტაპი უკავშირდება სილაბუსში გავლილ თემებს.

ეტაპი 1 — C# და OOP

კლასები, კონსტრუქტორები, მეთოდები, თვისებები, კოლექციები, Generics, მემკვიდრეობითობა, ინტერფეისები, Exceptions და LINQ.

ეტაპი 2 — მონაცემთა ბაზა

MS SQL Server, ცხრილები, Primary Key, Foreign Key, კავშირები, CRUD ოპერაციები, Views და Stored Procedures.

ეტაპი 3 — არქიტექტურა და მონაცემთა წვდომა

SOLID, Dependency Injection, Entity Framework Core და სურვილისამებრ ADO.NET-ის გამოყენება კონკრეტული ოპერაციისთვის.

ეტაპი 4 — ASP.NET Core Web API

Controllers, HTTP მეთოდები, CRUD, `IActionResult` , Dependency Injection და `HttpClient` .

ეტაპი 5 — ხარისხი და ტესტირება

ასინქრონული მეთოდები, Exceptions, ვალიდაცია, ლოგიკური შემოწმებები, Git და პროექტის დემონსტრირება.

2. მონაცემთა ბაზის სტრუქტურა

პროექტისთვის უნდა შეიქმნას მონაცემთა ბაზა `OnlineStoreDb`.

მონაცემთა ბაზა უნდა შედგებოდეს მინიმუმ ოთხი ძირითადი ცხრილისგან.

2.1. Products — პროდუქტები

ველი	ტიპი	დანიშნულება
`Id`	`int`	პროდუქტის უნიკალური იდენტიფიკატორი
`Name`	`nvarchar(100)`	პროდუქტის დასახელება
`Description`	`nvarchar(500)`	აღწერა
`Price`	`decimal(18,2)`	ფასი
`StockQuantity`	`int`	მარაგის რაოდენობა
`CategoryId`	`int`	კატეგორიის იდენტიფიკატორი

მოთხოვნები:

- პროდუქტის სახელი სავალდებულოა.
- ფასი უნდა იყოს ნულზე მეტი.
- მარაგის რაოდენობა არ უნდა იყოს უარყოფითი.
- პროდუქტი უნდა უკავშირდებოდეს არსებულ კატეგორიას.

2.2. Categories — კატეგორიები

ველი	ტიპი	დანიშნულება
`Id`	`int`	კატეგორიის იდენტიფიკატორი
`Name`	`nvarchar(100)`	კატეგორიის დასახელება
`Description`	`nvarchar(300)`	აღწერა

მაგალითად: კომპიუტერები, ტელეფონები, აქსესუარები.

ერთ კატეგორიაში შეიძლება ბევრი პროდუქტი შედიოდეს.

2.3. Customers — მომხმარებლები

ველი	ტიპი	დანიშნულება
`Id`	`int`	მომხმარებლის იდენტიფიკატორი
`FirstName`	`nvarchar(50)`	სახელი
`LastName`	`nvarchar(50)`	გვარი
`Email`	`nvarchar(150)`	ელფოსტა

`Phone`	`nvarchar(30)`	ტელეფონი

მოთხოვნები:

- სახელი და გვარი სავალდებულოა.
- ელფოსტა უნდა იყოს სწორ ფორმატში.
- ელფოსტა უნდა იყოს უნიკალური.

2.4. Orders — შეკვეთები

ველი	ტიპი	დანიშნულება
`Id`	`int`	შეკვეთის იდენტიფიკატორი
`CustomerId`	`int`	მომხმარებლის იდენტიფიკატორი
`OrderDate`	`datetime2`	შეკვეთის თარიღი
`Status`	`nvarchar(30)`	შეკვეთის სტატუსი
`TotalAmount`	`decimal(18,2)`	შეკვეთის ჯამური ღირებულება

შეკვეთასა და პროდუქტებს შორის საჭიროა დამატებითი ცხრილი `OrderItems`, რადგან ერთ შეკვეთაში შეიძლება რამდენიმე პროდუქტი იყოს, ხოლო ერთი პროდუქტი სხვადასხვა შეკვეთაში მონაწილეობდეს.

2.5. OrderItems — შეკვეთის პროდუქტები

ველი	ტიპი	დანიშნულება
`Id`	`int`	ჩანაწერის იდენტიფიკატორი
`OrderId`	`int`	შეკვეთის იდენტიფიკატორი
`ProductId`	`int`	პროდუქტის იდენტიფიკატორი
`Quantity`	`int`	რაოდენობა
`UnitPrice`	`decimal(18,2)`	შეკვეთის მომენტში არსებული ფასი

`UnitPrice` ინახავს ფასს შეკვეთის შექმნის მომენტისთვის. თუ პროდუქტის ფასი მოგვიანებით შეიცვლება, ძველი შეკვეთის ღირებულება არ უნდა შეიცვალოს.

ცხრილებს შორის კავშირები

Categories

1 კატეგორია

Products

N პროდუქტი

ერთი კატეგორია → ბევრი პროდუქტი

Customers

1 მომხმარებელი

Orders

N შეკვეთა

OrderItems

შეკვეთის პროდუქტები

Products

თითოეული პროდუქტი

3. პროექტის არქიტექტურა

სტუდენტმა უნდა გამოიყენოს ფენებად დაყოფილი არქიტექტურა, რათა ბიზნესლოგიკა, მონაცემთა წვდომა და HTTP მოთხოვნების დამუშავება ერთმანეთისგან განცალკევებული იყოს.

პროექტის რეკომენდებული სტრუქტურა:

```
OnlineStore/
│
├── OnlineStore.API/
│   ├── Controllers/
│   │   ├── ProductsController.cs
│   │   ├── CategoriesController.cs
│   │   ├── CustomersController.cs
│   │   └── OrdersController.cs
│   │
│   ├── DTOs/
│   ├── Middleware/
│   ├── Program.cs
│   └── appsettings.json
│
├── OnlineStore.Core/
│   ├── Entities/
│   ├── Interfaces/
│   └── Exceptions/
│
├── OnlineStore.Infrastructure/
│   ├── Data/
│   ├── Repositories/
│   └── Services/
│
└── README.md
```

ეს არის რეკომენდებული სტრუქტურა და არა ერთადერთი სწორი ვარიანტი. შეგიძლიათ პროექტის უფრო მარტივად ორგანიზებაც.

ფენების დანიშნულება:

- `API` — იღებს HTTP მოთხოვნებს და აბრუნებს HTTP პასუხებს.
- `Core` — შეიცავს ძირითად მოდელებს, ინტერფეისებსა და ბიზნესწესებთან დაკავშირებულ ტიპებს.
- `Infrastructure` — შეიცავს `DbContext`-ს, მონაცემთა ბაზასთან წვდომასა და შესაბამის იმპლემენტაციებს.
- `DTOs` — განსაზღვრავს, რა მონაცემებს იღებს და აბრუნებს API.
- `Repositories` და `Services` — გამოიყენება მონაცემებზე წვდომისა და ბიზნესლოგიკის ორგანიზებისთვის.

4. C# კლასების მოთხოვნები

სტუდენტმა უნდა შექმნას შემდეგი ძირითადი კლასები:

```
public class Product
{
    public int Id { get; set; }

    public string Name { get; set; } = string.Empty;

    public string Description { get; set; } = string.Empty;

    public decimal Price { get; set; }

    public int StockQuantity { get; set; }

    public int CategoryId { get; set; }

    public Category? Category { get; set; }
}
```

ეს კლასი წარმოადგენს პროდუქტს და შემდგომ გამოიყენება Entity Framework Core-ის საშუალებით მონაცემთა ბაზასთან მუშაობისას.

კატეგორიის მოდელი:

```
public class Category
{
    public int Id { get; set; }

    public string Name { get; set; } = string.Empty;

    public string Description { get; set; } = string.Empty;

    public ICollection<Product> Products { get; set; }
```

```
            = new List<Product>();
    }
```

შეკვეთის მოდელი:

```
    public class Order
    {
        public int Id { get; set; }

        public int CustomerId { get; set; }

        public DateTime OrderDate { get; set; }

        public string Status { get; set; } = "Pending";

        public decimal TotalAmount { get; set; }

        public ICollection<OrderItem> Items { get; set; }
            = new List<OrderItem>();
    }
```

სტუდენტმა თავად უნდა შექმნას `Customer` და `OrderItem` კლასებიც და განსაზღვროს მათ შორის საჭირო კავშირები.

მოთხოვნაა, რომ კლასები სწორად ასახავდეს მონაცემთა ბაზის ურთიერთობებს და გამოყენებული იყოს თვისებები, შესაბამისი ტიპები და კოლექციები.

5. Web API-ის მოთხოვნები

სტუდენტმა უნდა შექმნას REST-ის პრინციპებთან თავსებადი API, რომელიც გამოიყენებს შესაბამის HTTP მეთოდებს.

5.1. პროდუქტების მართვა

HTTP მეთოდი	Endpoint	დანიშნულება
GET	`/api/products`	ყველა პროდუქტის მიღება

GET	`/api/products/{id}`	ერთი პროდუქტის მიღება
POST	`/api/products`	ახალი პროდუქტის დამატება
PUT	`/api/products/{id}`	პროდუქტის განახლება
DELETE	`/api/products/{id}`	პროდუქტის წაშლა

5.2. კატეგორიების მართვა

HTTP მეთოდი	**Endpoint**	**დანიშნულება**
GET	`/api/categories`	კატეგორიების მიღება
GET	`/api/categories/{id}`	ერთი კატეგორიის მიღება
POST	`/api/categories`	კატეგორიის დამატება
PUT	`/api/categories/{id}`	კატეგორიის განახლება
DELETE	`/api/categories/{id}`	კატეგორიის წაშლა

5.3. მომხმარებლების მართვა

HTTP მეთოდი	Endpoint	დანიშნულება
GET	`/api/customers`	მომხმარებლების სიის მიღება
GET	`/api/customers/{id}`	მომხმარებლის მიღება
POST	`/api/customers`	მომხმარებლის დამატება
PUT	`/api/customers/{id}`	მომხმარებლის განახლება
DELETE	`/api/customers/{id}`	მომხმარებლის წაშლა

5.4. შეკვეთების მართვა

HTTP მეთოდი	Endpoint	დანიშნულება
GET	`/api/orders`	ყველა შეკვეთის მიღება
GET	`/api/orders/{id}`	შეკვეთის დეტალების მიღება

POST	`/api/orders`	შეკვეთის შექმნა
PUT	`/api/orders/{id}/status`	შეკვეთის სტატუსის განახლება
DELETE	`/api/orders/{id}`	შეკვეთის წაშლა, თუ ეს დაშვებულია ბიზნესწესებით

შეკვეთის შექმნისას API-მ უნდა გადაამოწმოს, არსებობს თუ არა მომხმარებელი და ყველა მითითებული პროდუქტი, საკმარისია თუ არა მარაგი და სწორია თუ არა შეკვეთილი რაოდენობები.

ჯამური ღირებულება უნდა გამოითვალოს სერვერზე. კლიენტის მიერ გამოგზავნილი `TotalAmount` არ უნდა ჩაითვალოს სანდო მონაცემად.

6. Entity Framework Core და SQL Server

სტუდენტმა უნდა:

1. შექმნას `DbContext`.
2. განსაზღვროს `DbSet` თითოეული ძირითადი მოდელისთვის.
3. დააკონფიგურიროს Primary Key-ები და Foreign Key-ები.
4. განსაზღვროს აუცილებელი ველები და მაქსიმალური სიგრძეები.
5. შექმნას Migration-ები.
6. შექმნას მონაცემთა ბაზა Migration-ების გამოყენებით.
7. განახორციელოს მონაცემების დამატება, მიღება, განახლება და წაშლა.
8. გამოიყენოს LINQ ფილტრაციისა და მონაცემების მოძიებისთვის.

მაგალითი:

```
public class OnlineStoreDbContext : DbContext
{
    public OnlineStoreDbContext(
        DbContextOptions<OnlineStoreDbContext> options)
        : base(options)
    {
    }
```

```
        public DbSet<Product> Products => Set<Product>();

        public DbSet<Category> Categories => Set<Category>();

        public DbSet<Customer> Customers => Set<Customer>();

        public DbSet<Order> Orders => Set<Order>();

        public DbSet<OrderItem> OrderItems => Set<OrderItem>();
    }
```

SQL Server-ის connection string უნდა განთავსდეს `appsettings.json`-ში და `DbContext` უნდა დარეგისტრირდეს Dependency Injection-ის გამოყენებით.

კონკრეტული `Program.cs` და პაკეტების კონფიგურაცია უნდა შეესაბამებოდეს პროექტში შერჩეულ .NET ვერსიას.

7. SOLID პრინციპების გამოყენება

სტუდენტმა უნდა აჩვენოს SOLID პრინციპების პრაქტიკული გამოყენება და არა მხოლოდ მათი განმარტება.

პრინციპი	პროექტში გამოყენება
Single Responsibility	Controller, Service და Repository სხვადასხვა პასუხისმგებლობას ასრულებენ
Open/Closed	ახალი ბიზნესლოგიკის დამატება შესაძლებელია არსებული კლასების

	მინიმალური ცვლილებით
Liskov Substitution	ინტერფეისის იმპლემენტაციები უნდა იყოს ურთიერთჩანაცვლებადი კონტრაქტის დარღვევის გარეშე
Interface Segregation	ინტერფეისები უნდა შეიცავდეს მხოლოდ შესაბამის პასუხისმგებლობებს
Dependency Inversion	მაღალი დონის ლოგიკა დამოკიდებული უნდა იყოს აბსტრაქციებზე და არა კონკრეტულ იმპლემენტაციებზე

მაგალითად, Controller-მა უნდა გამოიყენოს `IProductService` და არა თავად შექმნას კონკრეტული სერვისი `new ProductService()`-ის საშუალებით.

```
public interface IProductService
{
    Task<IReadOnlyList<Product>> GetAllAsync();

    Task<Product?> GetByIdAsync(int id);
}
```

ინტერფეისის რეალიზაცია უნდა დაარეგისტრიროს აპლიკაციამ Dependency Injection-ის საშუალებით.

8. ასინქრონულობა და Exceptions

მონაცემთა ბაზასთან მუშაობისას სტუდენტმა უნდა გამოიყენოს ასინქრონული EF Core მეთოდები, მაგალითად:

- `ToListAsync()`
- `FirstOrDefaultAsync()`
- `SaveChangesAsync()`
- `AnyAsync()`

ასევე უნდა გამოიყენოს `async` და `await` იმ ოპერაციებში, რომლებიც ასინქრონულად სრულდება.

შეცდომების დამუშავების მოთხოვნები:

- არარსებული პროდუქტის შემთხვევაში დაბრუნდეს `404 Not Found`.
- არასწორი მონაცემების შემთხვევაში დაბრუნდეს `400 Bad Request`.
- დუბლირებული ელფოსტის შემთხვევაში დაბრუნდეს შესაბამისი შეცდომა.
- არასაკმარისი მარაგის შემთხვევაში შეკვეთა არ შეიქმნას.
- მოულოდნელი შეცდომები დამუშავდეს ცენტრალიზებულად, მაგალითად Exception Middleware-ით.

სტუდენტმა არ უნდა გამოიყენოს `try/catch` ყველა მეთოდში ერთი და იმავე კოდის გასამეორებლად.

სასურველია შეცდომების ერთიანი დამუშავება.

სტუდენტისთვის ჩასაბარებელი შედეგები

პროექტის ჩაბარებისას სტუდენტმა უნდა წარმოადგინოს:

1. GitHub რეპოზიტორია — პროექტის სრული წყაროს კოდით და მოწესრიგებული commit-ებით.
2. მონაცემთა ბაზა — SQL სკრიპტები ან EF Core Migration-ები.
3. README.md — პროექტის აღწერა, გამოყენებული ტექნოლოგიები და გაშვების ინსტრუქცია.
4. API-ის ტესტირების მაგალითები — Swagger UI-ის, Postman-ის ან სხვა HTTP კლიენტის გამოყენებით.
5. პროექტის დემონსტრირება — ძირითადი CRUD ოპერაციების, შეკვეთის შექმნისა და შეცდომების დამუშავების ჩვენება.
6. კოდის ახსნა — სტუდენტმა უნდა შეძლოს საკუთარი არქიტექტურის, SOLID პრინციპებისა და ასინქრონული ოპერაციების დასაბუთება.

შეფასების კრიტერიუმები — 100 ქულა

კრიტერიუმი	ქულა
C# კლასები, OOP და ბიზნესლოგიკა	15
MS SQL Server და მონაცემთა ბაზის ურთიერთობები	15
Entity Framework Core და CRUD ოპერაციები	15
ASP.NET Core Web API და HTTP სტატუსები	15
SOLID, DI და პროექტის არქიტექტურა	15
Async/await და Exceptions	10
HttpClient და გარე API	5

ვალიდაცია და დამატებითი ფუნქციები	5
Git, README და პროექტის პრეზენტაცია	5
სულ	100

შეფასებისას მნიშვნელოვანია არა მხოლოდ ის, მუშაობს თუ არა აპლიკაცია, არამედ ისიც, რამდენად სწორად არის ორგანიზებული კოდი და რამდენად კარგად ესმის სტუდენტს საკუთარი გადაწყვეტილებები.